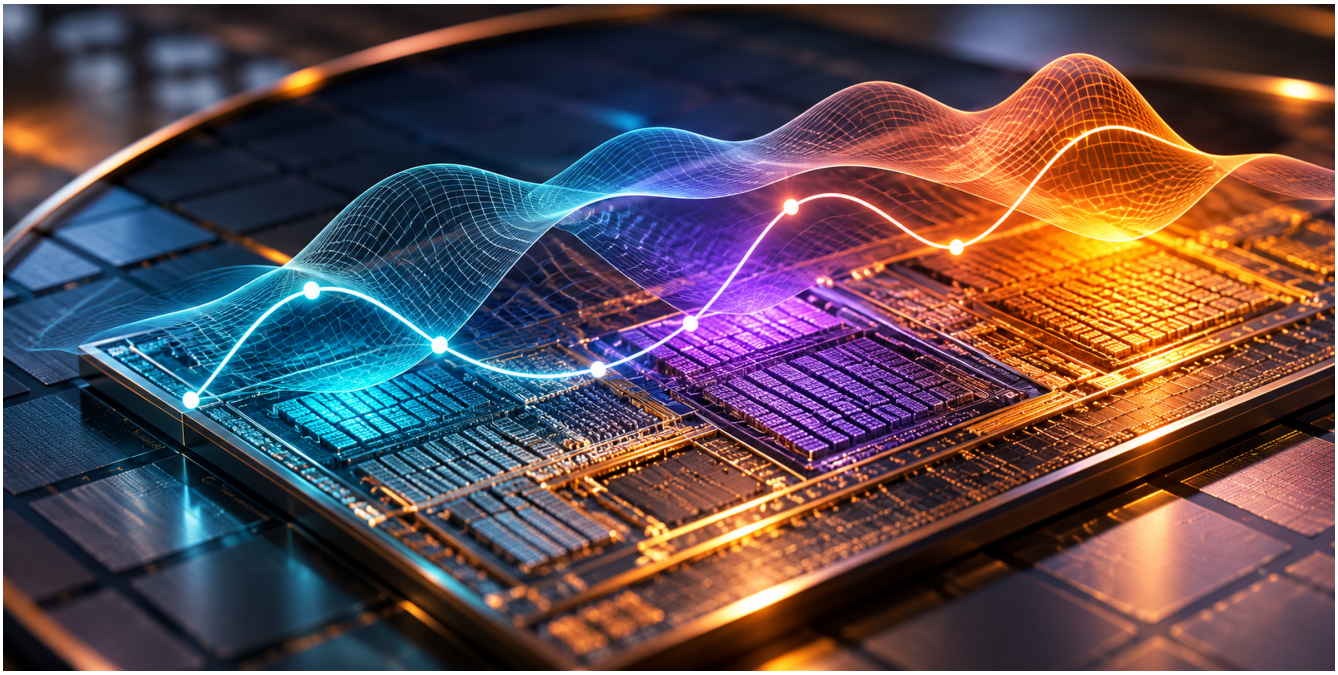
PACEVAL MATHEMATICAL ENGINE | INDUSTRIAL RESEARCH | WP 08

Programmable IC Timing and Leakage Corner Sweeps with paceval

Reusable mathematical execution graphs for versioned cell delay, standby leakage, and synthetic PVT screening



Cover: conceptual semiconductor visualization. It is not a computed research result, semiconductor photograph, or signoff report.

Research outcome

Original paceval MCP 4.26 executed 22 mathematical model graphs for an 81-corner voltage-temperature grid and 16 additional synthetic path scenarios. Core calculations, version comparisons, logical constraints and sensitivities were independently checked with high-precision mathematics. The study demonstrates defined model functionality, not foundry signoff or a local SDK runtime test.

Research scope	Status
22 substantive model graphs / 2,134 main graph-point evaluations	Remote paceval MCP 4.26 executed
1,162 ordinary stored results independently checked	Passed the specified numerical audit
24 selected TINC printed interval tests	Pointwise inclusion confirmed
C++ local shared-library execution and CTest	Not executed: compatible runtime unavailable

1. Industrial use case and current state of the art

Digital integrated-circuit timing depends on process variation, supply voltage, operating temperature, input transition, output load and circuit topology. An engineering team routinely evaluates cells and timing paths at multiple operating corners. Leakage consumption poses a different but related constraint, especially when voltage and temperature change.

The established workflow begins with cell and interconnect characterization, Liberty timing and power models, a netlist, clock and I/O constraints, and extracted parasitics. Static timing analysis (STA) propagates arrivals and required times through a timing graph. OpenSTA is a documented gate-level STA engine supporting Liberty, Verilog, SDC, SDF and SPEF. It can report checks for multiple modes and analysis corners [1, 2].

This research studies a narrower programmable component: explicit reusable mathematical formulas that score hypothetical cells and retain versioned screening rules over many input vectors. A host may then select corners for further examination within established EDA software. No claim is made that a closed-form cell-delay surrogate adequately characterizes a physical library arc or that repeated use of one cell models a routed circuit.

Recent developments relevant to the research boundary

On 12 March 2026 OpenSTA 3.0.1 added statistical timing analysis using Liberty Variation Format (LVF) data with normal and skew-normal probability models [3]. Such characterized variations are a distinct capability from deterministic parameter sweeps or pointwise floating-point error intervals. The ASAP7 predictive 7 nm PDK offers an academic basis for possible future open-source characterization experiments, but its models are not claimed to be tied to a manufacturing foundry [4, 5].

Hypothetical industrial application

A cell-library engineering team could register a transparent model for each selected cell family, evaluate a controlled voltage-temperature-load grid, change an explicit model term, and compare corner flags. A full timing engine would still supply the real topology, arc tables and final constraints. In the demonstrator the cell, path and budgets are deliberately synthetic; the scientific hypothesis concerns functionality and verification of mathematical graph execution.

This work reports no CPU, GPU, FPGA, ASIC, throughput, energy or economic comparisons. The main contribution is an explicit and auditable mathematical execution path, validated only for the stipulated model and test inputs.

2. Research question and architecture

Research question: can the original paceval engine register persistent, simultaneously available mathematical model versions and execute complete delay/power/score/guard chains across many PVT input vectors, with outputs independently checked against a separately implemented high-precision reference?

Figure 1 | Model screening architecture (concept illustration)



Figure 1. Conceptual functional architecture, not computed output. The production mathematics is executed by reusable paceval graphs; the host handles data and decisions.

Engine and host responsibilities

paceval creates and reuses computation handles for all 22 domain-related expressions, including combined path delay, timing slack, power margins, minimum-based final scores, a relational and logical acceptance guard, explicit analytical sensitivities and model deltas. The C++ host validates dimensional domains, parses CSV records, dispatches batches, selects worst returned results and writes audit files. Python and independently written native C++ functions are verification oracles only.

The remote research execution uses the available paceval MCP 4.26 service and its actual create, multi-graph and multi-scenario evaluation operations. The Linux C++ project uses the original paceval_main.h / paceval_main.cpp interfaces with an external ABI-compatible shared library. These are distinct pathways: a successful MCP run does not establish that the local SDK program compiled, linked and ran.

Reproducible evidence chain

Fixed synthetic CSV inputs → exactly declared and versioned formula strings → real paceval MCP graphs → graph-wise output arrays and error codes → retained numerical records → independent 90-digit checks → use-case-oriented diagrams. The corresponding per-function mapping is machine-readable in results/math_execution_inventory.csv, and exact counts are in results/execution_evidence.json.

3. Mathematical model, definitions and units

The original research proposal supplied a nonphysical but interpretable additive cell delay model and an exponential leakage current model. Version 2 makes these models explicitly distinct by adding voltage-dependent load response, temperature-load and slew-load coupling in delay, plus temperature-curvature and voltage-temperature interaction in leakage.

$$D_1 = t_0 + a_C C + a_S S + a_T(T - T_{ref}) + a_V(V_{ref}/V)^p$$

$$D_2 = D_1 + a_C C[(V_{ref}/V)^q - 1] + a_{CT} C(T - T_{ref}) + a_{CS} CS$$

$$I_1 = I_0\, e^{b_T(T - T_{ref}) + b_V(V - V_{ref})}$$

$$I_2 = I_0\, e^{b_T \Delta T + b_V \Delta V + b_{TT} \Delta T^2 + b_{VT} \Delta T \Delta V}$$

$$s_i = T_{req} - T_{arr} - N D_i - D_{wire}, \quad P_i = V I_i$$

$$g_i = \min(s_i,\ k(P_{cap} - P_i))$$

Typeset equations: model specification (not a computed dataset). D1 and D2 are per-cell delays. I1 and I2 are cell standby currents. The synthetic path uses N repeated identical cell delays, while k has units of ps/nW.

Symbol and range	Meaning and units
V 0.70 to 1.10; T −40 to 125	Supply voltage (V) and temperature (°C)
C, S	Output capacitance (fF), input slew (ps)
D, I, P	Cell delay (ps), standby current (nA), cell standby power (nW)
N, Dwire	Illustrative number of repeated cells, fixed path wire delay (ps)
Treq, Tarr	Required arrival constraint and initial arrival time (ps)
Pcap, k	Single-cell leakage power budget (nW) and score scale (ps/nW)

The model is evaluated only after checking V > 0, nonnegative load and slew, finite parameters, positive budget and legal exponent ranges. These bounds also avoid invalid fractional-power and exponential domains. Timing guard equality is non-strict at zero margin. The Boolean graph result is 1 when all listed mathematical constraints hold, otherwise 0.

4. Actual paceval mathematical execution

The important experimental test is not the number of registered expressions but which useful mathematical end-results were computed through the original engine. The following groups all ran within actual remote MCP graphs for the synthetic scenario datasets.

Function group	Executed engine path / version
Per-cell delay, load and temperature response	paceval: delay_v1_ps, delay_v2_ps
Current and static power	paceval: leak_v1_nA, leak_v2_nA, leak_power_v1_nW, leak_power_v2_nW
Repeated-cell path and timing slack	paceval: path_delay_v1/v2_ps, slack_v1/v2_ps
Leakage budget margins and combined score	paceval: leak_margin_v1/v2_nW, composite_score_v1/v2_ps
Logical all-conditions constraint	paceval: corner_guard_v1, corner_guard_v2
Analytical delay and leakage sensitivities	paceval: delay_sensitivity_voltage/temp_v2, leak_sensitivity_temp_v2
Versioned mathematical differences	paceval: model_delta_delay_ps, model_delta_power_nW
Normalized leakage consumption	paceval: leakage_power_fraction_v2
Input safety, record sequencing, I/O and selection	C++ host, not claimed as paceval mathematics
Independent numerical comparison	Native C++ and 90-digit Python mpmath; reference only

The engine natively evaluates arithmetic, powers and exponentials, plus infix min and mixed relational/Boolean expressions. Separate MCP operator tests verified the selected grammar against signed and mathematical modulo, min/max, AND/OR/NOT/XOR, and boundary semantics. The full list of actual graph strings and version identifiers is in src/formula_manifest.json. No symbolic automatic differentiation claim is made; sensitivity formulas were derived explicitly.

Model versions are simultaneous immutable graph handles. The host may compare outputs or select a worst corner but does not reconstruct the domain-level score after receiving a trivial engine subtotal.

5. Deterministic workload and executed evidence

All operating points use synthetic values. The 81-case grid comprises 9 voltage settings from 0.70 to 1.10 V and 9 temperatures from −40 to 120 °C. It fixes capacitance at 12 fF and slew at 45 ps. A separate 16-case set varies voltage, temperature, load and slew to exercise the same graph versions. These are independent scenarios, not a dynamic transistor or temperature trajectory.

Measured activity or verification	Observed fact
Distinct substantive paceval model graphs created/executed	22 / 22
MCP model graph-point evaluations across main batches	2,134 = 22 × (81 + 16)
Numerical results preserved in current research archive	1,162 = 10 × 81 + 22 × 16
Independently checked preserved ordinary values	1,162 passed using 90-digit mpmath
Additional TINC-enabled graph-point evaluations	24; 24 printed inclusions confirmed
Operator-syntax checks through the same remote engine	21 of 21 passed
Local Linux paceval shared-library execution, CTest	Not executed

Concrete version-sensitive industrial screening example

At synthetic V = 1.10 V and T = 100 °C (load 12 fF, input slew 45 ps), paceval v2 calculated per-cell delay 59.6882 ps, path slack +109.2470 ps, and standby power 122.3307 nW. The assumed per-cell budget is 120 nW. The composite minimum score was therefore negative and the Boolean guard returned 0. The v1 guard at that same point returned 1.

Within the 81 tested corners, v1 marked 79 corners mathematically acceptable under its stipulated rules, whereas v2 marked 76. Three grid corners changed their Boolean status between versions. These statistics refer only to the synthetic grid and do not constitute a continuous admissible silicon process envelope.

The archive separates evaluated graph-point combinations from stored values; the grid file contains ten selected outputs from each full 22-graph evaluation. The 16-case snapshot file retains every one of the 22 graph values per case. All published values can be reproduced from the recorded files.

6. Versioned cell-delay behavior

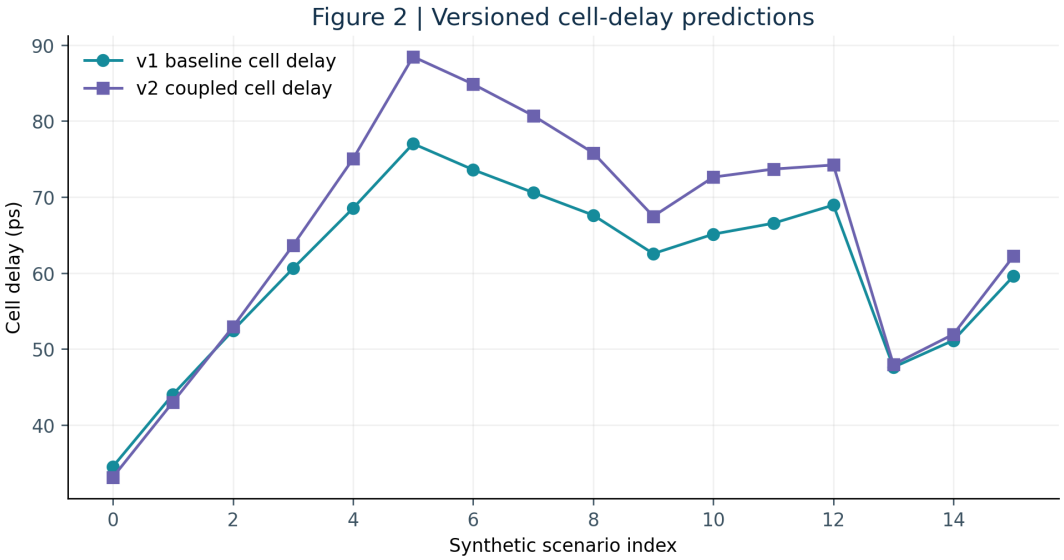


Figure 2. Real paceval outputs for two simultaneous cell-delay models across 16 independently specified synthetic scenarios.

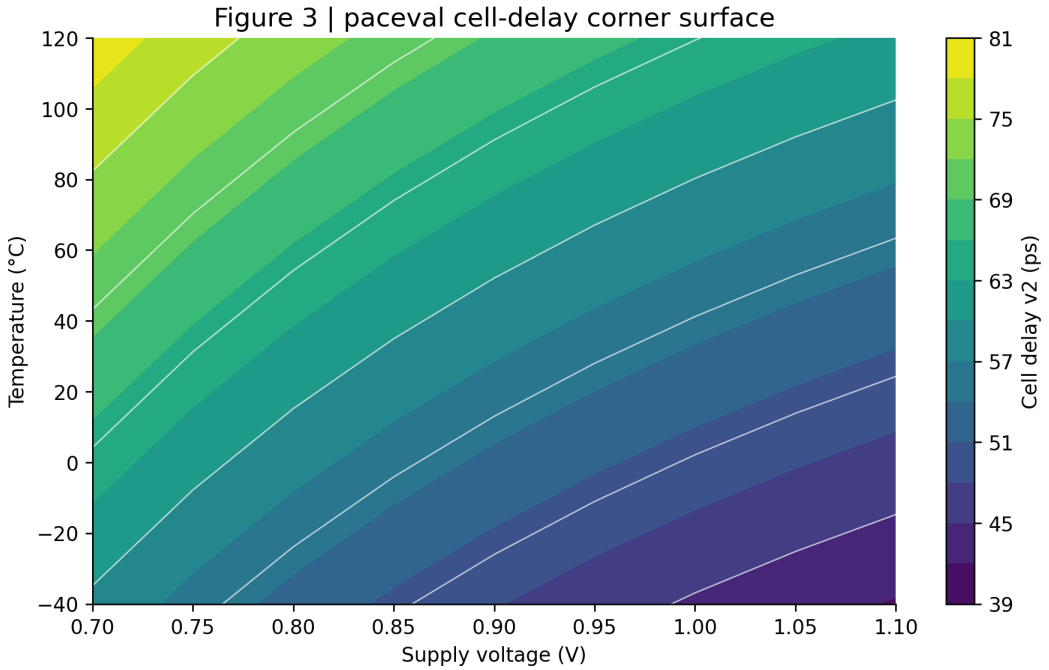


Figure 3. Actual paceval model-v2 delay values on the 81-point PVT grid. Smooth contours between sampled values are a visualization aid, not additional engine calculations.

The versioned formulas change the dependence on voltage, load, input slew and temperature. The numeric difference is an output of the actual version-delta paceval graph, not a comparison of measured foundry-cell characterization accuracy.

7. Cell leakage power and synthetic limits

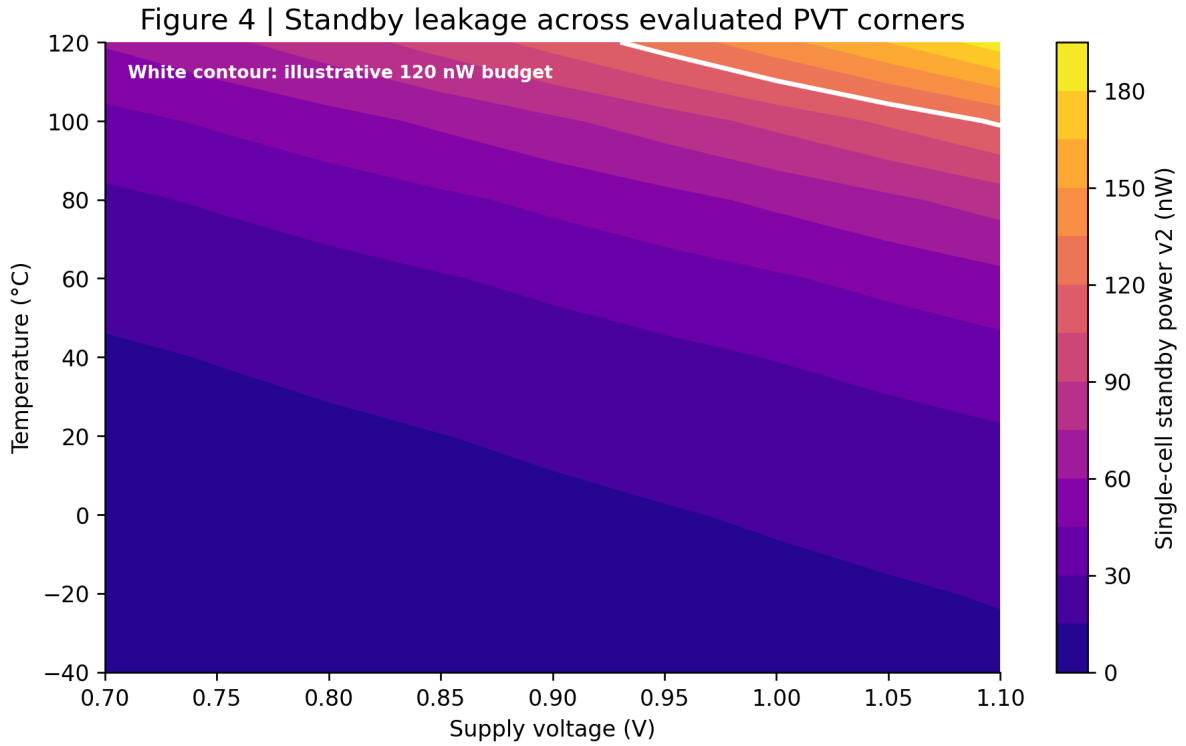


Figure 4. Recorded model-v2 per-cell static leakage across PVT corners. The white isoline represents the illustrative 120 nW budget, interpolated for display.

The exponential current model is intentionally explicit and reproducible. Actual leakage in a characterized standard-cell library is state-dependent and process-dependent, so this model must not be treated as a physical library power table. The research question concerns whether a versioned nonlinear mathematical expression and its bound can be evaluated and audited consistently.

Selected synthetic corner	Modeled power v2	Illustrative criterion
1.05 V, 100 °C	107.577 nW	Below 120 nW
1.10 V, 100 °C	122.331 nW	Above 120 nW
0.95 V, 120 °C	126.409 nW	Above 120 nW
1.10 V, 120 °C	187.866 nW	Above 120 nW

A positive timing slack alone is insufficient to satisfy a combined timing and leakage screen. This separation is one of the central useful domain-level computations executed inside paceval.

8. Composed score and Boolean operating-corner rules

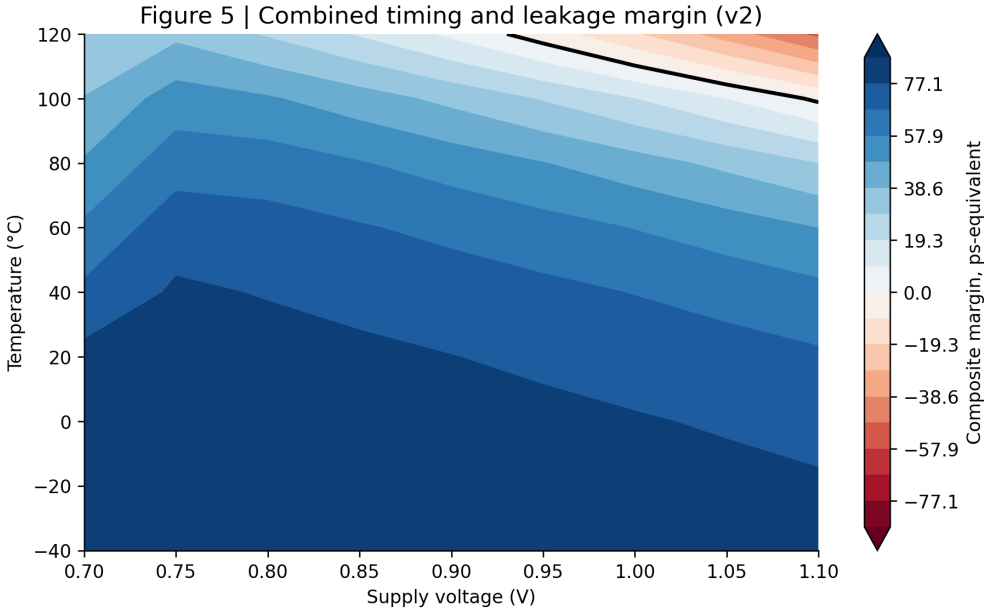
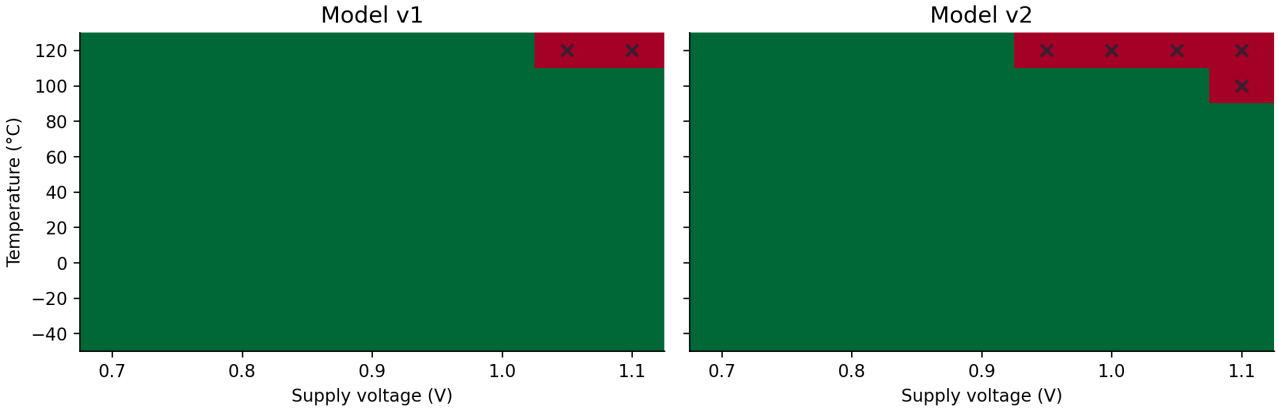


Figure 5. paceval combined minimum-based score, where zero distinguishes the stipulated synthetic pass and fail regions.

Figure 6 | Synthetic accept/review masks returned by paceval



X = corner fails at least one modeled timing, leakage or domain rule

Figure 6. Actual 0/1 paceval guard results for unchanged v1 and v2 graphs, plotted over the same 81 input corners.

The score expresses two different numeric quantities through an explicitly stated ps/nW scale, while the companion guard checks Boolean timing, leakage and domain conditions. The C++ host is responsible for any further REVIEW/REJECT policy; paceval does not grant manufacturing or design approval.

9. Timing-path and leakage scenario examples

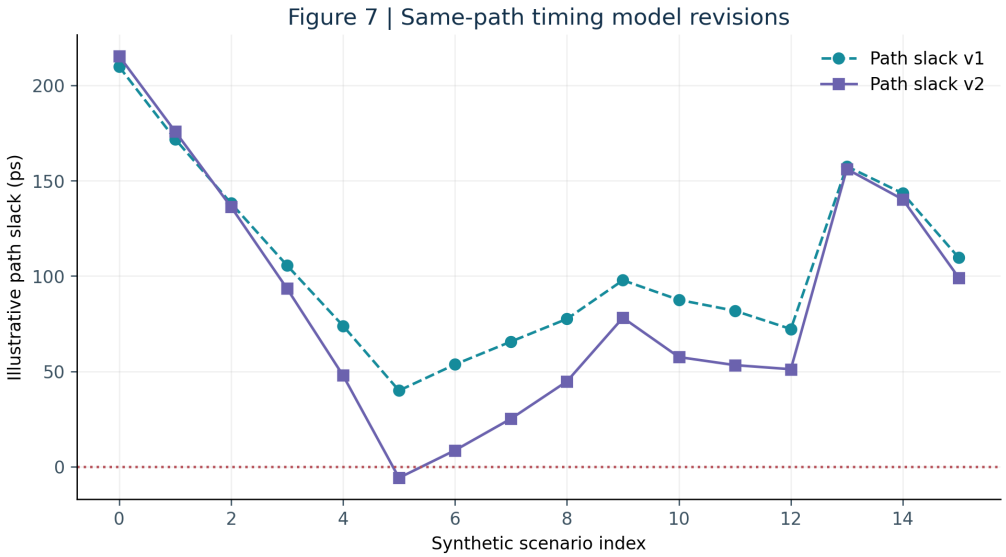


Figure 7. Model-specific illustrative repeated-cell path slack from real paceval outputs. These paths are intentionally not netlist-level STA.

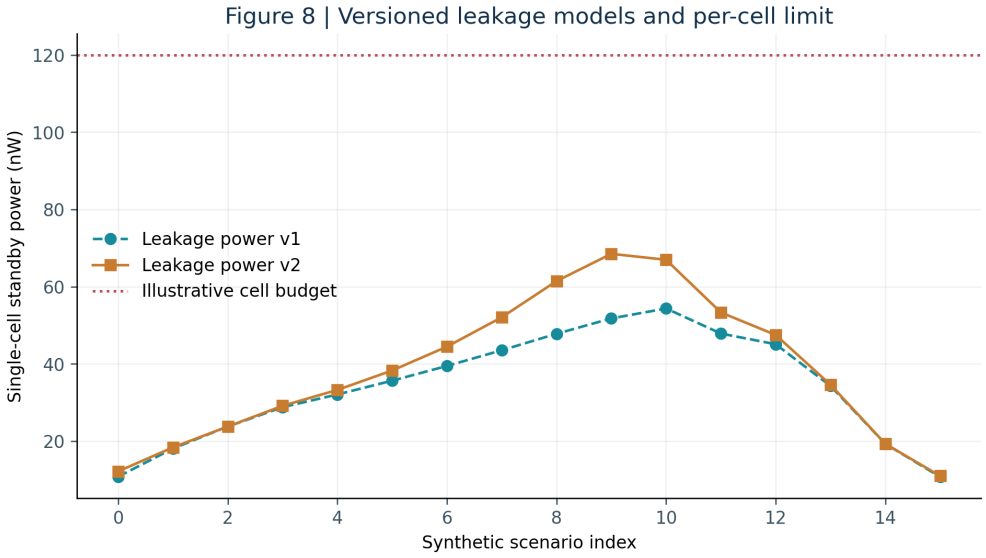


Figure 8. Corresponding actual paceval standby power from two immutable model versions on the same scenario records.

For synthetic scenario S05, model v1 computed positive timing slack while v2 computed negative slack. This demonstrates that a controlled model change can alter a mathematically significant engineering flag, without establishing either model as a better semiconductor physics representation.

10. Model-revision surface and sensitivity

Figure 9 | paceval model-revision delay surface

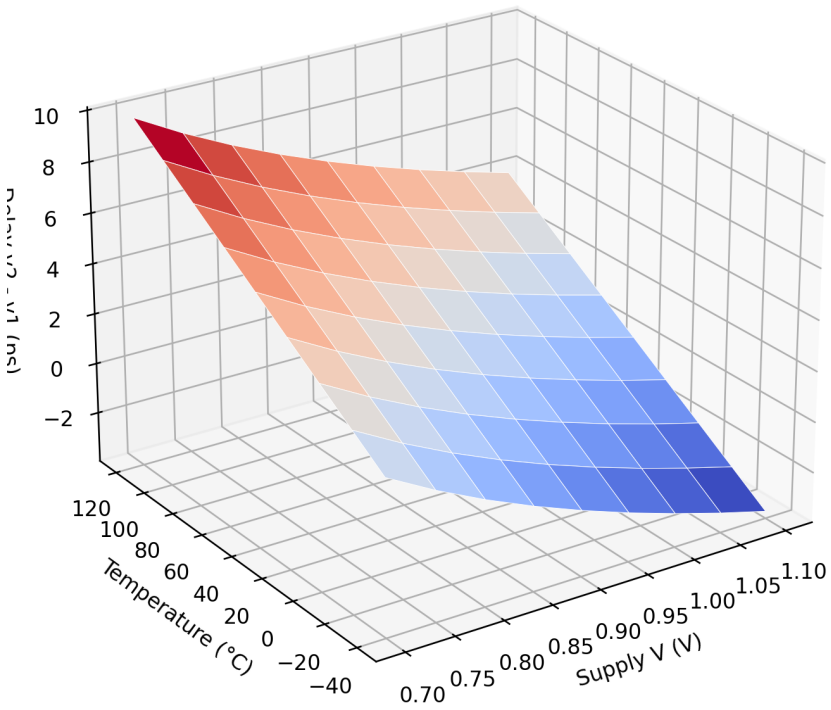


Figure 9. Difference between v2 and v1 cell delays, computed by a dedicated reusable paceval formula across 81 actual corner points.

The v2 voltage sensitivity is explicitly represented as −aC·C·q·(Vref/V)^q/V − aV·p·(Vref/V)^p/V in ps/V. The temperature sensitivity equals aT + aCT·C in ps/K. The v2 current sensitivity to temperature is I2 times bT + 2bTT·ΔT + bVT·(V−Vref), in nA/K. These were actual separately registered paceval expressions and independently verified for the 16 retained scenarios.

These sensitivities describe the chosen parametric surrogate. They are not automatic derivatives supplied by a generic differentiation API, physical transistor response coefficients, or globally certified monotonicity results. The host controls the parameter sweep and stores values without rewriting the mathematical graph versions.

11. Independent verification and pointwise TINC output

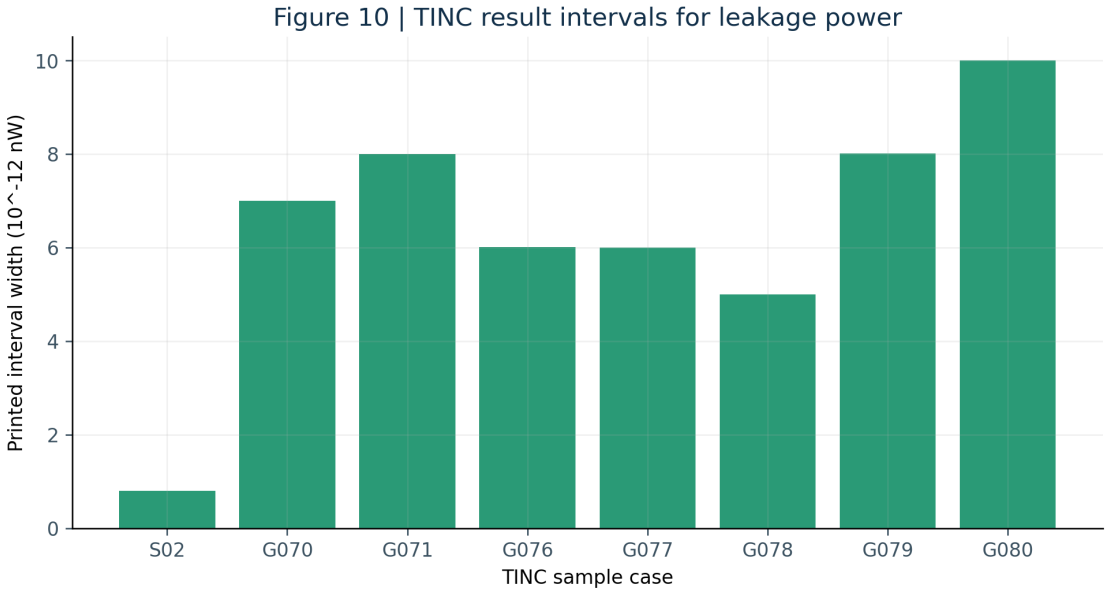


Figure 10. Widths of the printed upper and lower paceval TINC bounds at eight selected operating cases. The endpoints are actual remote service outputs.

A separately coded mpmath reference with 90 decimal digits reconstructed each input as its exact IEEE-754 binary64 fraction. The audit checked all 1,162 stored ordinary model outputs. The application-scaled tolerance in the technical file was satisfied in every recorded case. Exact numerical differences are intentionally retained only in results/high_precision_audit.json, not printed here.

For the eight additional selected cases, three interval-enabled versions of slack, leakage power and the final composite score produced 24 pointwise TINC interval outputs. All 24 printed decimal intervals enclosed their independently evaluated exact-binary-input mathematical references. The remote engine reported no per-point error for those test vectors.

This is a finite-sample inclusion test, not a proof of globally valid interval arithmetic, sound enclosure of all physically uncertain V/T values, or certified circuit signoff. The distinction between floating-point rounding bounds, characterization variance (including Liberty LVF), model inadequacy and hardware measurement uncertainty is essential.

Operator and data safety boundaries

A separate remote test also executed 21 operator cases, confirming the expected difference between the two modulo conventions and the Boolean 0/1 behavior of selected composite expressions. The C++ CSV parser rejects expressions before invoking the original numeric string converter; this parser and the numeric-output routine are present and syntax-checked, but no local SDK runtime execution is asserted.

12. Scientific interpretation, reproducibility and references

What is established

The original paceval MCP server successfully registered and reused versioned expressions for all the listed semiconductor-domain mathematical functions and evaluated them over the supplied synthetic datasets. Persistently handled math graphs produced independently checked values, distinct model decisions and representative TINC bounds. The C++ source defines original SDK and CSV integration and passes syntax-only compilation.

What remains outside the evidence

This study does not contain real Liberty timing arcs, measured silicon, SPICE or path-aware waveform simulation, cell rise/fall asymmetry, setup/hold constraints, extracted RC parasitics, voltage droop, IR heating, multi-mode STA, LVF delay distributions or foundry signoff. The four-stage path is a mathematical demonstration only. The compatible Linux shared library was absent, so there are no CTest run results and no local SDK conversion execution. Future work may use openly licensed Liberty reference tables with careful model-to-table and EDA comparisons.

Reproduction commands

From the project root, run **python3 validation/validate_records.py** for the independent reference, **python3 scripts/make_figures.py** for all result diagrams and **python3 src/check_readme_math.py** for structural GitHub markdown and local link checks. For full original SDK integration, provide an ABI-compatible shared library and run **bash scripts/build_and_test.sh /absolute/path/libpaceval_linux_sharedLIB.so**. No binary paceval library is shipped.

Selected external references

[1] OpenSTA architecture, OpenROAD STA documentation.

[2] OpenSTA, Multi-corner scenes and statistical timing examples.

[3] OpenSTA, Release 3.0.1, 12 March 2026 (LVF statistical analysis).

[4] Arizona State University, ASAP7 predictive process design kit.

[5] Vashishtha et al., ASAP7: A 7-nm finFET predictive process design kit, Microelectronics Journal 53 (2016). DOI.

[6] paceval, Original SDK API and documented operators. The remote service reported v4.26.

Conclusion: the synthetic, versioned paceval mathematical workflow is functionally demonstrated and independently checked. An EDA deployment or chip-signoff claim requires an entirely separate validation program.